

Name : Saran .S

Reg no: 192473003

Course name: CSA0402 – Java programming

C01 AT3 ASSESSMENT

CODING CHALLENGE

Q1 Smart University Payroll Management System

25 marks

A university employs three categories of staff: Teaching Faculty, Technical Staff, and Visiting Faculty. The payroll system should calculate the monthly salary based on the employee type while maintaining a common employee hierarchy. Requirements Create a superclass Employee with the following private data members: employeeId employeeName department basicSalary Provide: Default constructor Parameterized constructor Getter and Setter methods displayEmployee() calculateSalary() Create the following subclasses: TeachingFaculty teachingHours incentivePerHour Salary: basicSalary + (teachingHours × incentivePerHour) TechnicalStaff overtimeHours overtimeRate Salary: basicSalary + (overtimeHours × overtimeRate) VisitingFaculty lecturesHandled paymentPerLecture Salary: lecturesHandled × paymentPerLecture Override calculateSalary() in all subclasses. Create an array of Employee objects and demonstrate runtime polymorphism. Display: Employee details Monthly salary Highest-paid employee Average salary Search an employee using the employee ID.

Test Case 1 (Normal Case)

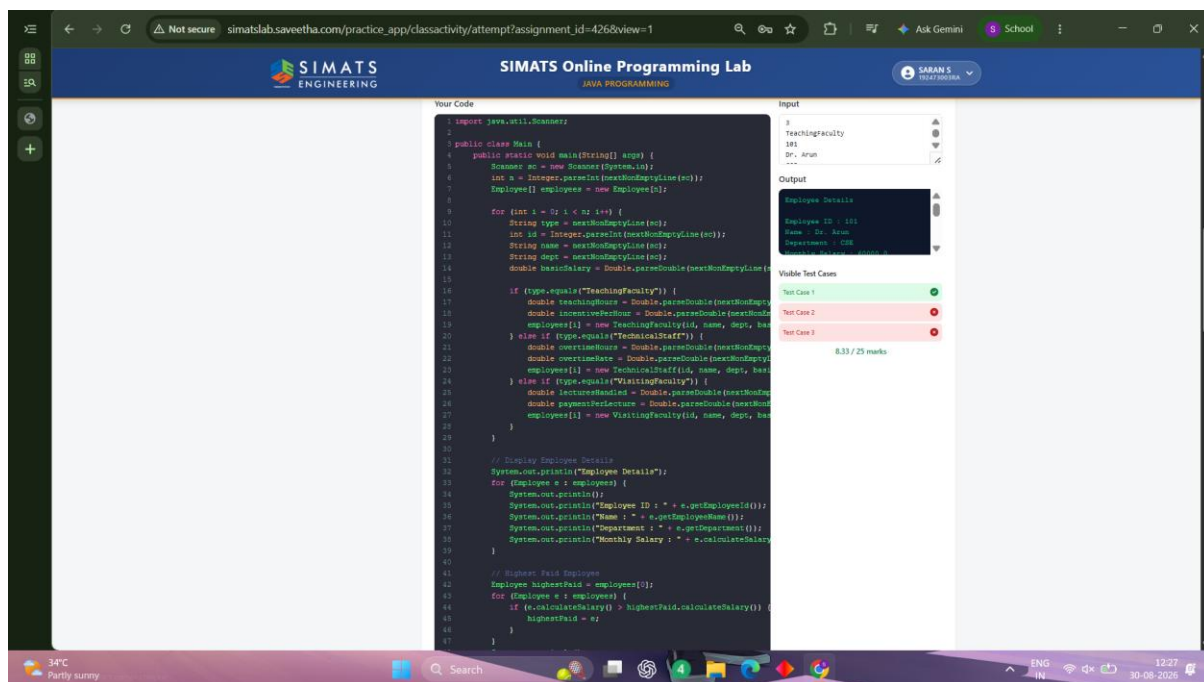
Input

3

TeachingFaculty

101

Dr. Arun



Q2 Smart Vehicle Insurance Claim System

25 marks

Problem Statement An insurance company processes claims for three categories of vehicles: Car, Bike, and Truck. The claim amount depends on the vehicle type. **Requirements** Superclass: Vehicle Private members: vehicleNumber ownerName insuredAmount Subclasses: Car Bike Truck Each subclass overrides calculateClaimAmount() using different formulas. The application should: Read details for N vehicles Store them using an array of superclass references Calculate claim amounts using runtime polymorphism Search by vehicle number Display the vehicle with the highest claim amount **Question 3 (Very Hard)** Online Shopping Order Processing System (50 Marks) **Problem Statement** An e-commerce company sells products in three categories: Electronics Clothing Grocery The billing process varies for each category. **Requirements** Superclass: Product Private members: productId productName price quantity Subclasses: Electronics Clothing Grocery Each subclass overrides calculateBill(). The program should: Store products in an array of superclass references Calculate the final bill using runtime polymorphism Search products by ID Display the most expensive order Compute total revenue

The screenshot shows the SIMATS Online Programming Lab interface. The main area displays a Java program for processing vehicle insurance claims. The program includes a Scanner for input, a Vehicle array, and a switch statement to handle different vehicle types (Car, Bike, Truck). The output shows the claim amounts for each vehicle.

```

import java.util.Scanner;

public class Main {
    static Scanner sc = new Scanner(System.in);

    static String nextNonEmptyLine() {
        String line;
        do {
            line = sc.nextLine().trim();
        } while (line.isEmpty());
        return line;
    }

    public static void main(String[] args) {
        int n = Integer.parseInt(nextNonEmptyLine());

        Vehicle[] vehicles = new Vehicle[n];

        for (int i = 0; i < n; i++) {
            String type = nextNonEmptyLine();
            String vehicleNumber = nextNonEmptyLine();
            String ownerName = nextNonEmptyLine();
            double insuredAmount = Double.parseDouble(nextNonEmptyLine());

            switch (type) {
                case "Car":
                    vehicles[i] = new Car(vehicleNumber, ownerName, insuredAmount);
                    break;
                case "Bike":
                    vehicles[i] = new Bike(vehicleNumber, ownerName, insuredAmount);
                    break;
                case "Truck":
                    vehicles[i] = new Truck(vehicleNumber, ownerName, insuredAmount);
                    break;
            }
        }

        for (int i = 0; i < vehicles.length; i++) {
            Vehicle v = vehicles[i];
            System.out.println("Vehicle : " + v.getVehicleNumber());
            System.out.println("Claim Amount : " + v.calculateClaimAmount());
            System.out.println();
        }

        Vehicle highest = vehicles[0];
        for (int i = 1; i < vehicles.length; i++) {
            Vehicle v = vehicles[i];
            if (v.calculateClaimAmount() > highest.calculateClaimAmount()) {
                highest = v;
            }
        }

        System.out.println("Highest Claim Vehicle:");
        System.out.println(highest.getVehicleNumber());
        System.out.println(highest.calculateClaimAmount());
        System.out.println();

        String maybeLabel = nextNonEmptyLine();
        String searchNumber = nextNonEmptyLine();
        if (maybeLabel.toLowerCase().contains("search")) {
            searchNumber = nextNonEmptyLine();
        } else {
            searchNumber = maybeLabel;
        }

        boolean found = false;
        for (Vehicle v : vehicles) {
            if (v.getVehicleNumber().equalsIgnoreCase(searchNumber)) {
                System.out.println("Owner : " + v.getOwnerName());
                System.out.println("Claim Amount : " + v.calculateClaimAmount());
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("Vehicle Not Found");
        }
    }
}

abstract class Vehicle {
    String vehicleNumber;
    String ownerName;
    double insuredAmount;

    abstract double calculateClaimAmount();
}

class Car extends Vehicle {
    Car(String vehicleNumber, String ownerName, double insuredAmount) {
        this.vehicleNumber = vehicleNumber;
        this.ownerName = ownerName;
        this.insuredAmount = insuredAmount;
    }

    @Override
    double calculateClaimAmount() {
        return insuredAmount * 0.5;
    }
}

class Bike extends Vehicle {
    Bike(String vehicleNumber, String ownerName, double insuredAmount) {
        this.vehicleNumber = vehicleNumber;
        this.ownerName = ownerName;
        this.insuredAmount = insuredAmount;
    }

    @Override
    double calculateClaimAmount() {
        return insuredAmount * 0.3;
    }
}

class Truck extends Vehicle {
    Truck(String vehicleNumber, String ownerName, double insuredAmount) {
        this.vehicleNumber = vehicleNumber;
        this.ownerName = ownerName;
        this.insuredAmount = insuredAmount;
    }

    @Override
    double calculateClaimAmount() {
        return insuredAmount * 0.7;
    }
}

```

The output shows the claim amounts for each vehicle:

```

Vehicle : 7610481234
Claim Amount : 640000.0
Vehicle : 7610481234
Claim Amount : 640000.0

```

The visible test cases show a score of 8.33 / 25 marks.

The screenshot shows the continuation of the Java program. The program includes a Scanner for input, a Vehicle array, and a switch statement to handle different vehicle types (Car, Bike, Truck). The output shows the claim amounts for each vehicle.

```

import java.util.Scanner;

public class Main {
    static Scanner sc = new Scanner(System.in);

    static String nextNonEmptyLine() {
        String line;
        do {
            line = sc.nextLine().trim();
        } while (line.isEmpty());
        return line;
    }

    public static void main(String[] args) {
        int n = Integer.parseInt(nextNonEmptyLine());

        Vehicle[] vehicles = new Vehicle[n];

        for (int i = 0; i < n; i++) {
            String type = nextNonEmptyLine();
            String vehicleNumber = nextNonEmptyLine();
            String ownerName = nextNonEmptyLine();
            double insuredAmount = Double.parseDouble(nextNonEmptyLine());

            switch (type) {
                case "Car":
                    vehicles[i] = new Car(vehicleNumber, ownerName, insuredAmount);
                    break;
                case "Bike":
                    vehicles[i] = new Bike(vehicleNumber, ownerName, insuredAmount);
                    break;
                case "Truck":
                    vehicles[i] = new Truck(vehicleNumber, ownerName, insuredAmount);
                    break;
            }
        }

        for (int i = 0; i < vehicles.length; i++) {
            Vehicle v = vehicles[i];
            System.out.println("Vehicle : " + v.getVehicleNumber());
            System.out.println("Claim Amount : " + v.calculateClaimAmount());
            System.out.println();
        }

        Vehicle highest = vehicles[0];
        for (int i = 1; i < vehicles.length; i++) {
            Vehicle v = vehicles[i];
            if (v.calculateClaimAmount() > highest.calculateClaimAmount()) {
                highest = v;
            }
        }

        System.out.println("Highest Claim Vehicle:");
        System.out.println(highest.getVehicleNumber());
        System.out.println(highest.calculateClaimAmount());
        System.out.println();

        String maybeLabel = nextNonEmptyLine();
        String searchNumber = nextNonEmptyLine();
        if (maybeLabel.toLowerCase().contains("search")) {
            searchNumber = nextNonEmptyLine();
        } else {
            searchNumber = maybeLabel;
        }

        boolean found = false;
        for (Vehicle v : vehicles) {
            if (v.getVehicleNumber().equalsIgnoreCase(searchNumber)) {
                System.out.println("Owner : " + v.getOwnerName());
                System.out.println("Claim Amount : " + v.calculateClaimAmount());
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("Vehicle Not Found");
        }
    }
}

abstract class Vehicle {
    String vehicleNumber;
    String ownerName;
    double insuredAmount;

    abstract double calculateClaimAmount();
}

class Car extends Vehicle {
    Car(String vehicleNumber, String ownerName, double insuredAmount) {
        this.vehicleNumber = vehicleNumber;
        this.ownerName = ownerName;
        this.insuredAmount = insuredAmount;
    }

    @Override
    double calculateClaimAmount() {
        return insuredAmount * 0.5;
    }
}

class Bike extends Vehicle {
    Bike(String vehicleNumber, String ownerName, double insuredAmount) {
        this.vehicleNumber = vehicleNumber;
        this.ownerName = ownerName;
        this.insuredAmount = insuredAmount;
    }

    @Override
    double calculateClaimAmount() {
        return insuredAmount * 0.3;
    }
}

class Truck extends Vehicle {
    Truck(String vehicleNumber, String ownerName, double insuredAmount) {
        this.vehicleNumber = vehicleNumber;
        this.ownerName = ownerName;
        this.insuredAmount = insuredAmount;
    }

    @Override
    double calculateClaimAmount() {
        return insuredAmount * 0.7;
    }
}

```

The output shows the claim amounts for each vehicle:

```

Vehicle : 7610481234
Claim Amount : 640000.0
Vehicle : 7610481234
Claim Amount : 640000.0

```

The visible test cases show a score of 8.33 / 25 marks.

```
300     }
301     this.insuredAmount = insuredAmount;
302     }
303     public String getVehicleNumber() {
304         return vehicleNumber;
305     }
306     public String getOwnerName() {
307         return ownerName;
308     }
309     public double getInsuredAmount() {
310         return insuredAmount;
311     }
312     public abstract double calculateClaimAmount();
313 }
314
315 class Car extends Vehicle {
316     public Car(String vehicleNumber, String ownerName, double insuredAmount) {
317         super(vehicleNumber, ownerName, insuredAmount);
318     }
319     @Override
320     public double calculateClaimAmount() {
321         return getInsuredAmount() * 0.80;
322     }
323 }
324
325 class Bike extends Vehicle {
326     public Bike(String vehicleNumber, String ownerName, double insuredAmount) {
327         super(vehicleNumber, ownerName, insuredAmount);
328     }
329     @Override
330     public double calculateClaimAmount() {
331         return getInsuredAmount() * 0.60;
332     }
333 }
334
335 class Truck extends Vehicle {
336     public Truck(String vehicleNumber, String ownerName, double insuredAmount) {
337         super(vehicleNumber, ownerName, insuredAmount);
338     }
339     @Override
340     public double calculateClaimAmount() {
341         return getInsuredAmount() * 0.50;
342     }
343 }
```